

OMIS 105

Meet Marimo

Your notebook for writing SQL with DuckDB

Agenda

- What Marimo is
- Cells and reactivity
- Starting a notebook
- Running SQL with DuckDB
- The four rules we always follow
- Common mistakes

What Is Marimo?

A **notebook**: a page where you write a little code, run it, and see the answer right away.

👉 We use it to write **SQL** and see the **table** that comes back.

Why a Notebook?

You could write SQL in a plain file and run the whole thing.

A notebook lets you work **one question at a time**:

- Write one query
- See the result
- Ask the next question

A Notebook Is Made of Cells

A **cell** is one small box of code.

- One cell = one idea
- Cells run top to bottom
- Each cell shows its own result

The Big Idea: Reactivity

Change one cell, and **every cell that depends on it updates by itself.**

No "run this cell, then that cell, then remember to re-run the first one."

👉 The notebook keeps itself correct.

Why That Matters

In older notebooks you could:

1. Run a cell
2. Change it
3. Forget to re-run what came after

Now the numbers on screen do not match the code.

👉 Marimo does not let this happen.

A Marimo Notebook Is a `.py` File

Open `marimo_101_duckdb_sql.py` in any text editor and you will see plain Python.

That means it works with normal tools — email it, put it in a folder, track it in Git.

Starting Marimo

Open a terminal in the folder that holds the notebook, then:

```
marimo edit marimo_101_duckdb_sql.py
```

It opens in your **web browser**.

Stopping Marimo

Go back to the terminal window and press:

```
Ctrl + C
```

👉 Closing the browser tab does **not** stop it.

Two Kinds of Cells

Cell	Holds	You use it for
Python / SQL	Code	Queries and results
Markdown	Text	Notes and headings

Now: SQL with DuckDB

Marimo runs Python. DuckDB is the database.

Every notebook starts by opening a **connection** to DuckDB:

```
import duckdb  
  
con = duckdb.connect(database=":memory:")
```

What **:memory:** Means

The database lives in memory only.

- It is built fresh every time you run the notebook
- It disappears when you close it
- Nothing is saved to disk

👉 Perfect for practice. Nothing to clean up.

Rule 1 — Use `con.execute()`

This is how **every** notebook in this course runs SQL:

```
con.execute("SELECT * FROM products")
```

👉 One pattern, all quarter.

Rule 2 — Add `.fetchdf()` to See Results

Creating a table shows nothing:

```
con.execute("CREATE OR REPLACE TABLE products (...)"
```

Asking a question ends with `.fetchdf()`:

```
con.execute("SELECT * FROM products").fetchdf()
```

Rule 3 — Pass `con` Into the Cell

Every cell that touches the database takes `con` :

```
def _(con):  
    con.execute("SELECT * FROM products").fetchdf()
```

👉 This is what tells Marimo the cell depends on the database.

Rule 4 — CREATE OR REPLACE TABLE

Always write:

```
CREATE OR REPLACE TABLE products (...)
```

Not just CREATE TABLE .

👉 So you can run the cell twice without an error.

Putting It Together

```
import duckdb
con = duckdb.connect(database=":memory:")
```

```
con.execute("""
    CREATE OR REPLACE TABLE products (
        product_id    INTEGER,
        product_name   VARCHAR,
        price          DECIMAL(6,2)
    )
""")
```

And Then Ask a Question

```
con.execute("""
    SELECT    product_name, price
    FROM      products
    WHERE     price > 20
    ORDER BY  price DESC
""").fetchdf()
```

👉 The result appears as a table, right under the cell.

Comments Inside SQL

Use `--`, not `#`:

```
con.execute("""
    -- only the expensive items
    SELECT * FROM products WHERE price > 20
""").fetchdf()
```

Making It Interactive

Marimo can add real controls:

```
limit = mo.ui.slider(1, 20, value=5, label="How many rows?")
```

Move the slider → the query re-runs → the table updates.

👉 No code change. Just drag.

Common Mistakes

You see	The fix
Cell runs, no table appears	Add <code>.fetchdf()</code>
Table with name ... does not exist	Run the <code>CREATE</code> cell first
... already exists	Use <code>CREATE OR REPLACE TABLE</code>
marimo: command not found	<code>python3 -m marimo edit <file></code>

Quick Reference

Task	Command
Open a notebook	<code>marimo edit <file>.py</code>
Stop Marimo	<code>Ctrl + C</code> in the terminal
Connect to DuckDB	<code>duckdb.connect(database=":memory:")</code>
Run SQL	<code>con.execute("...")</code>
See the result	<code>.fetchdf()</code>

What to Remember

1. Cells are small; each one shows its own result
2. Change one cell — the rest update themselves
3. `con.execute(...)` runs the SQL
4. `.fetchdf()` shows the table
5. `Ctrl + C` stops the notebook

Your Turn

Open a terminal in `tutorials/marimo_introduction/` and run:

```
marimo edit marimo_101_duckdb_sql.py
```

Then work through it. Change a number. Watch what updates.

Questions?

Bring them to office hours — see

`course_information/QUESTIONS_and_OFFICE_HOURS.md` .

OMIS 105 — Introduction to Database Management Systems — Fall 2026